

Chapter One

September 2025

This paper only contains the formal parts of the chapter, no comments are given here.

1 Syntax of the Programming Language

Arithmetic Expressions

$A := x$	a variable of the program, can be any identifier
$ 0 1 \dots$	any integer numbers
$ A + A A * A \dots$	operations
$ f(V, V, \dots)$	function call, f can be any identifier, the number or parameters is arbitrary but fixed

Boolean Expressions

$B := A \leq A \dots$	comparisons
$ B \text{ and } B B \text{ or } B \dots$	connectors
$ f(V, V, \dots)$	function call

Values

$V := A | B$

Instructions

$I := x := V$	assignement
$ I; I$	sequence
$ \text{if } B \text{ then } I \text{ else } I \text{ endif}$	
$ \text{while } B \text{ do } I \text{ done}$	

Function declarations and Programs

$F := f(x_1, \dots, x_n) =$ $I; V$	a function with n parameters is an instruction followed by the return value
$P := (F)^+$	a program is a non-empty list of functions

2 Semantics of the Programming Language

Semantics for Arithmetic Expressions

$$\begin{aligned}
 Val(x, \mu) &= \mu(x) & (variable) \\
 Val(c, \mu) &= c & (constante) \\
 Val(a_1 + a_2, \mu) &= Val(a_1, \mu) + Val(a_2, \mu) \\
 &\dots
 \end{aligned}$$

Semantics for Boolean Expressions

$$\begin{aligned}
 Val(a_1 \leq a_2, \mu) &= \mathbf{tt} \text{ iff } Val(a_1, \mu) \leq Val(a_2, \mu) \\
 Val(b_1 \text{ and } b_2, \mu) &= \mathbf{tt} \text{ iff } Val(b_1, \mu) = \mathbf{tt} \text{ and } Val(b_2, \mu) = \mathbf{tt} \\
 &\dots
 \end{aligned}$$

Semantics for Instructions

$$\begin{aligned}
 Mem(x := t, \mu) &= \mu[x \leftarrow Val(t, \mu)] \\
 Mem(i_1; i_2, \mu) &= Mem(i_2, Mem(i_1, \mu)) \\
 Mem(\text{if } b \text{ then } i_1 \text{ else } i_2 \text{ endif}, \mu) &= Mem(i_1, \mu) \text{ if } Val(b, \mu) = \mathbf{tt} \\
 &\quad Mem(i_2, \mu) \text{ otherwise} \\
 Mem(\text{while } b \text{ do } i \text{ done}, \mu) &= \mu \text{ if } Val(b, \mu) = \mathbf{ff} \\
 &\quad Mem(\text{while } b \text{ do } i \text{ done}, Mem(i, \mu)) \text{ otherwise}
 \end{aligned}$$

Semantics for Functions

$$\begin{aligned}
 Val(i; v, \mu) &= Val(v, Mem(i, \mu)) \\
 Val(f(t_1, \dots, t_n), \mu) &= Val(i; v, \mu[x_1 \leftarrow Val(t_1, \mu), \dots, x_n \leftarrow Val(t_n, \mu)])
 \end{aligned}$$

3 The WP Computation

No Loop

$$\begin{aligned}
 \mathbf{WP}(x := t, Q) &= Q[x \leftarrow t] \\
 \mathbf{WP}(i_1; i_2, Q) &= \mathbf{WP}(i_1, \mathbf{WP}(i_2, Q)) \\
 \mathbf{WP}(\text{if } b \text{ then } i_1 \text{ else } i_2 \text{ endif}, Q) &= b \Rightarrow \mathbf{WP}(i_1, Q) \wedge \neg b \Rightarrow \mathbf{WP}(i_2, Q)
 \end{aligned}$$

Adding Loops

$$\begin{aligned}
 \mathbf{WP}(\text{while } b \text{ do } c \text{ done}, Q) &= I \text{ if} \\
 &\quad \text{if } \forall \mu, \mu \models b \wedge I \Rightarrow \mathbf{WP}(c, I) \\
 &\quad \text{and } \forall \mu, \mu \models \neg b \wedge I \Rightarrow Q
 \end{aligned}$$